

Section 84 — Master Dynamic Structures Mixed Contest

Contest contract

Six new problems combine value-domain navigation, two-sided beats, reversible hashes, dynamic forest aggregates, and Kruskal reconstruction trees.

Problem	Main pressure point
A	carry zero-side sums while navigating a quantile
B	keep symmetric min/max extrema consistent
C	swap forward and reverse hashes under lazy reversal
D	distinguish represented paths from auxiliary splay structure
E	an internal reconstruction node records the exact union time
F	compose affine lazy tags in chronological order on an exposed path

Recognition map

Operation pattern	Boundary to retain
static subarray order statistic and prefix sum	wavelet level interval plus skipped sums
range clamps and sums	two extreme classes at each segment node
editable sequence with reversal	implicit order and two directional hashes
online forest path updates	exposed preferred path in a link-cut tree
earliest threshold when vertices connect	Kruskal reconstruction ancestry

Discovery walkthrough: Earliest Connection

Process edges by increasing time or weight. When an edge joins two DSU components, create a new reconstruction node with those component roots as children and label it by the current threshold. Leaves are original vertices. The LCA of two leaves is the exact merge event that first placed them in one component, so its label answers the query.

This converts a whole history of DSU partitions into one static tree. Equal weights must be handled according to whether all edges at that weight become available simultaneously; careless sequential parentage can change tree shape but must not change the stored threshold answer.

Discovery walkthrough: Reversible String Hash

A segment summary needs length, forward hash, and reverse hash. Concatenation uses powers to shift one child. Lazy reversal swaps children and swaps forward with reverse; it does not recompute characters. Equality of a segment with its reverse then becomes equality of the two hashes, preferably under two moduli or another collision policy stated by the problem.

Proof and validation checklist

- Wavelet A: do skipped zero blocks contribute both count and sum correctly?
- Beats B: are min and max invariants updated symmetrically?
- Treap C: does reversal swap every directional aggregate?
- Link-cut D/F: is the requested represented path exposed before applying tags?
- Reconstruction E: does one internal node mean one genuine DSU merge event?

Use sorted slices, literal arrays/strings, adjacency-list forests, and replayed edge prefixes as independent tiny oracles. Random operation sequences are more effective than isolated samples for exposing stale lazy tags and auxiliary- root mistakes.

Exercises

A–F form the complete mixed contest.